Chapter 10: Searching and Sorting

Technical / interview questions, exercises, multiple choice questions and answer key for Chapter 10 (Searching and Sorting) of Data Structures.

Programming Exercises

1. Write a C program to implement Linear Search on an unsorted array.
2. Implement Binary Search using recursion.
3. Write a program to perform Interpolation Search on a uniformly distrib uted sorted array.
4. Implement Jump Search and count the number of comparisons made.
5. Compare execution time of Linear Search and Binary Search for large input sizes.
6. Write a program to sort an array using Bubble Sort.
7. Modify Bubble Sort to stop early if the array becomes sorted.
8. Implement Insertion Sort and trace each pass.
9. Write a program to sort student records using Selection Sort based on marks.
10. Implement Merge Sort using divide-and-conquer approach.
11. Modify Merge Sort to count number of comparisons.
12. Write a program to implement Quick Sort using the first element as pivot.
13. Modify Quick Sort to use random pivot selection.
14. Implement Heap Sort and show the heap construction process.
15. Write a program to implement Radix Sort for integers.
16. Implement Shell Sort using gap sequence n/2, n/4, ..., 1.
17. Write a program to perform Tree Sort using Binary Search Tree.
18. Compare performance of Quick Sort and Merge Sort for random data.
19. Write a program to sort strings alphabetically using Insertion Sort.
20. Implement a program that checks whether an array is already sorted.
21. Write a menu-driven program to perform different searching techniques.
22. Write a menu-driven program to perform different sorting techniques.
23. Count number of swaps in Bubble Sort for best and worst case.
24. Compare Jump Search and Interpolation Search for large datasets.
25. Implement Hybrid Sort (Insertion + Merge Sort).
26. Write a program to search an element in a sorted linked list.
27. Implement Quick Sort without recursion (using stack).
28. Analyze time complexity of all sorting algorithms using code experimentation.

Technical / Interview Questions

1. What is the difference between searching and sorting?
2. When is Interpolation Search preferred over Binary Search?
3. Why does Jump Search use √n step size?
4. Explain best, average, and worst-case time complexity of Quick Sort.
5. Why is Merge Sort stable?
6. Why is Heap Sort not stable?
7. What is the main drawback of Bubble Sort?
8. How does Insertion Sort perform on nearly sorted data?

9. Why is Selection Sort rarely used in practice?
10. Explain the divide-and-conquer strategy in Merge Sort.
11. What is pivot selection in Quick Sort? Why is it important?
12. What is the space complexity of Merge Sort?
13. Why is Quick Sort generally faster than Merge Sort in practice?
14. Explain heap property in Heap Sort.
15. When is Radix Sort preferred over comparison-based sorting?
16. What is the worst-case complexity of Shell Sort?
17. How does Tree Sort work internally?
18. What is a stable sorting algorithm? Give examples.
19. Which sorting algorithms are in-place?
20. Compare Jump Search and Binary Search.
21. What are internal and external sorting techniques?
22. How does data distribution affect Interpolation Search?
23. What is time complexity of searching in unsorted array?
24. What are advantages of Quick Sort?
25. What is the difference between comparison-based and non-comparison sorting?
26. Why is Radix Sort linear time?
27. Explain the concept of h-sorting in Shell Sort.
28. Which sorting algorithm is best for small datasets?

Multiple Choice Questions

1. The time complexity of Linear Search in worst case is:

(a) O(log n)
(b) O(n)
(c) O(n log n)
(d) O(1)

2. Binary Search requires:

(a) Sorted array
(b) Unsorted array
(c) Linked list
(d) Stack

3. Interpolation Search works best when:

(a) Data is random
(b) Data is uniformly distributed
(c) Data is reverse sorted
(d) Data is duplicated

4. Jump Search uses step size approximately equal to:

(a) n
(b) log n
(c) √n
(d) n^2

5. Best case of Bubble Sort occurs when:

(a) Array is sorted
(b) Array is reverse sorted
(c) Array has duplicates
(d) Array size is small

6. Which sorting algorithm is stable?

(a) Heap Sort
(b) Quick Sort
(c) Merge Sort
(d) Selection Sort

7. Merge Sort follows:

(a) Greedy strategy
(b) Divide and Conquer
(c) Dynamic Programming
(d) Backtracking

8. Average case time complexity of Quick Sort is:

(a) $O(n^2)$
(b) O(log n)
(c) O(n log n)
(d) O(n)

9. Heap Sort is based on:

(a) Queue
(b) Stack
(c) Heap data structure
(d) Graph

10. Radix Sort is:

(a) Comparison-based
(b) Non-comparison based
(c) Recursive
(d) Divide and conquer

11. Worst case of Quick Sort occurs when:

(a) Pivot is middle
(b) Pivot is smallest or largest
(c) Data is random
(d) Data is uniform

12. Insertion Sort is efficient for:

(a) Large datasets
(b) Nearly sorted arrays
(c) Random arrays
(d) Linked lists only

13. Shell Sort improves:

(a) Selection Sort
(b) Merge Sort
(c) Insertion Sort
(d) Heap Sort

14. Tree Sort uses:

(a) AVL Tree
(b) Binary Search Tree
(c) Heap
(d) Graph

15. Time complexity of Selection Sort is:

(a) $O(n^2)$
(b) O(n log n)
(c) O(log n)
(d) O(n)

16. Binary Search has worst case complexity:

(a) O(n)
(b) O(n log n)
(c) O(log n)
(d) O(1)

17. Stable sorting maintains:

(a) Order of elements
(b) Position of duplicates
(c) Relative order of equal elements
(d) Sorted array

18. Space complexity of Merge Sort is:

(a) O(1)
(b) O(n)
(c) O(log n)
(d) $O(n^2)$

19. Heap Sort builds:

(a) Min Heap
(b) Max Heap
(c) Both
(d) None

20. Radix Sort processes digits from:

(a) MSB to LSB
(b) LSB to MSB
(c) Middle
(d) Random

21. Jump Search requires:

(a) Sorted data
(b) Unsorted data
(c) Tree
(d) Heap

22. Worst case time of Insertion Sort is:

(a) O(n)
(b) O(n log n)
(c) $O(n^2)$
(d) O(log n)

23. Which is fastest for small arrays?

(a) Insertion Sort
(b) Merge Sort
(c) Heap Sort
(d) Radix Sort

24. Quick Sort is:

(a) Stable
(b) Not stable
(c) Linear
(d) Constant

25. Time complexity of Jump Search is:

(a) $O(\sqrt{n})$
(b) $O(n)$
(c) $O(\log n)$
(d) $O(n^2)$

26. Tree Sort worst case occurs when BST is:

(a) Balanced
(b) Complete
(c) Skewed
(d) Full

27. Selection Sort performs:

(a) Minimum swaps
(b) Maximum swaps
(c) No swaps
(d) Recursive swaps

28. Merge Sort is suitable for:

(a) Linked lists
(b) Arrays only
(c) Stack
(d) Queue

29. Shell Sort gap sequence ends with:

(a) 0
(b) 1
(c) 2
(d) n

30. Which algorithm has best average complexity?

(a) Bubble Sort
(b) Selection Sort
(c) Quick Sort
(d) Insertion Sort

Answer Key

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	B	2	A	3	B	4	C	5	A
6	C	7	B	8	C	9	C	10	B
11	B	12	B	13	C	14	B	15	A

16	C	17	C	18	B	19	B	20	B
21	A	22	C	23	A	24	B	25	A
26	C	27	A	28	A	29	B	30	C